

pandas + matplotlib — Reference Card

Series/DataFrame · selection · groupby · pivot/melt · merge · time-series · IO · dtypes · matplotlib. Stack: pandas ≥2.2 · matplotlib ≥3.8 · Python ≥3.10 · Modern: Copy-on-Write, PyArrow backend, method chaining

◆ Master pattern (every pandas script)

- 1. **Load** — read_parquet/csv/sql; declare dtypes explicitly
- 2. **Enable CoW** — pd.options.mode.copy_on_write = True (default in pd 3.x)
- 3. **Inspect** — .info() .describe() .head() .isna().sum()
- 4. **Clean** — dtypes, index, missing, duplicates
- 5. **Transform** — method chain:
.assign().query().groupby().agg()
- 6. **Persist** — to_parquet (columnar, typed, fast) > CSV

Golden rule (pd 3.x): chained assignment (df["a"][mask] = v) FAILS silently. Always use df.loc[mask, "a"] = v.

◆ Starter · notebook opener

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
pd.options.mode.copy_on_write = True      # default in pandas 3.x; opt-in 2.2+
pd.set_option("display.max_columns", 40)
pd.set_option("display.width", 180)
```

```
# Read with dtypes (parquet preserves them; CSV needs help)
df = pd.read_parquet("data.parquet")
# CSV: cast at read time (avoid inference surprises)
df = pd.read_csv("data.csv",
    parse_dates=["dt"],
    dtype={"id": "int32", "region": "category", "score": "float32"})
```

```
# Modern: PyArrow-backed dtypes (nullable, faster on strings)
df = pd.read_csv("data.csv", dtype_backend="pyarrow")
```

```
df.info(memory_usage="deep"); df.head()
```

● Series + DataFrame

Construction

```
s = pd.Series([1, 2, 3], index=["a","b","c"], name="score")
df = pd.DataFrame({"a": [1,2,3], "b": [4,5,6]},
    index=["r1","r2","r3"])
df = pd.DataFrame.from_records([{"a":1,"b":2}, {"a":3,"b":4}])
df = pd.DataFrame(np.arange(12).reshape(3,4),
    columns=list("abcd"))
```

Key attributes

```
df.shape; df.index; df.columns; df.dtypes
df.to_numpy()      # → numpy (df.values is LEGACY – type-unsafe)
df.T               # transpose
df.rename(columns={"a":"x"})      # never rely on inplace=
df.set_index("dt"); df.reset_index()
```

Vectorised ops (fast, always prefer)

```
df["gross"] = df["qty"] * df["price"]      # element-wise
df["z"] = (df["x"] - df["x"].mean()) / df["x"].std()
df["is big"] = df["gross"] > 100
# NEVER iterate rows – use .assign / vectorised expressions or .apply as last resort
```

● Selection (.loc / .iloc / query)

Task	Idiom
Single column	df["a"] (Series) · df[["a"]] (DataFrame)
Multi-column	df[["a","b"]]
Label rows	df.loc["r1"] · df.loc[["r1","r3"]]
Position rows	df.iloc[0] · df.iloc[:5]
Row + col slice	df.loc[m, ["a","b"]]
Boolean mask	df[df["x"] > 0]
Multi-cond	df[(df.a > 0) & (df.b < 5)]
Query string	df.query("a > 0 and b < 5")
NaN filter	df[df["a"].notna()]
Category filter	df[df["region"].isin(["EU","US"])]

```
Set via loc      df.loc[mask, "y"] = 0
```

Chained assignment trap: df["x"][mask] = v FAILS in pd 3.x (was warning in 2.x). Always df.loc[mask, "x"] = v.

▲ groupby + agg (the workhorse)

Single + multi aggregations

```
g = df.groupby("region")
g["gross"].sum()
g["gross"].agg(["sum","mean","std","count"])
g.agg({"gross":"sum", "qty":"mean"})      # different per column
g.agg(total=("gross","sum"), avg=("gross","mean"))      # named agg (recommended)
```

Multi-key + observed=

```
df.groupby(["region","segment"], observed=True).size()
# observed=True IS RECOMMENDED for categorical keys – pandas 3.x default
```

Transform (aligned back)

```
df["z_by_region"] = (df["x"]
    - df.groupby("region")["x"].transform("mean")
    ) / df.groupby("region")["x"].transform("std")
# transform returns a Series with the ORIGINAL index – perfect for feature eng
```

Apply (last resort)

```
df.groupby("region").apply(lambda g: g.nlargest(3, "gross"),
    include_groups=False)
# include_groups=False is the pd ≥2.2 default – silences the DeprecationWarning
```

Common patterns

Goal	Idiom
Top-N per group	df.groupby("g")["x"].nlargest(3)
Cumulative in-group	df.groupby("g")["x"].cumsum()
Rank in-group	df.groupby("g")["x"].rank(method="dense")
Fraction of total	x / df.groupby("g")["x"].transform("sum")
Filter groups	df.groupby("g").filter(lambda g: len(g) > 5)

▲ pivot / melt (long ↔ wide)

pivot_table (long → wide)

```
wide = df.pivot_table(index="date", columns="ticker",
    values="close",
    observed=True,
    aggfunc="mean", fill_value=0,
    # NOT df.pivot() – pivot_table handles duplicates via aggfunc)
```

melt (wide → long)

```
long = df.melt(id_vars=["date"],
    value_vars=["aapl","msft","goog"],
    var_name="ticker", value_name="close")
```

stack / unstack (MultiIndex ↔ wide)

```
df_multi = df.set_index(["region","segment"])
df_multi.unstack("segment")      # segment values become columns
df_multi.stack()      # in pd 3.x future_stack is default (kwargs removed)
```

crosstab (counts / contingency)

```
pd.crosstab(df["region"], df["segment"], normalize="index")
```

▲ merge / concat / join

merge — the SQL join

```
pd.merge(orders, customers, on="cust_id", how="left")      # ← default: how="inner"
pd.merge(a, b, left_on="a_id", right_on="b_id", how="outer",
    indicator=True, suffixes=("_a", "_b"))
# indicator=True adds "_merge" column: left_only / right_only / both
```

how=	Behaviour
inner	only matches (default)
left / right	keep left/right; NaN for missing
outer	keep both
cross	Cartesian product

merge_asof (nearest-key join — time-series critical)

```
pd.merge_asof(trades.sort_values("ts"),
    quotes.sort_values("ts"),
    on="ts", by="ticker", direction="backward",
    tolerance=pd.Timedelta("100ms"))
# for each trade, find most recent quote <= trade time within tolerance
```

concat (stack / column-bind)

```
pd.concat([df1, df2], axis=0, ignore_index=True)      # stack rows
pd.concat([df1, df2], axis=1)      # stack cols (align on index)
pd.concat([df1, df2], keys=["A","B"])      # add MultiIndex level
```

join (index-based shortcut)

```
a.join(b, how="left")      # b's index → a's rows; column-wise merge on index
```

◆ Time-series

DatetimeIndex — always set + asfreq

```
df["dt"] = pd.to_datetime(df["dt"], utc=True)
df = df.set_index("dt").sort_index()
df = df.asfreq("D")      # explicit freq – ESSENTIAL for resample/rolling
```

resample (change frequency)

```
df["y"].resample("D").mean()      # downsample
df["y"].resample("h").last().ffill()      # upsample + fill
df.resample("1W").agg({"y":"mean", "vol":"sum"})
# OHLC one-liner:
df["px"].resample("1D").ohlc()
```

Freq alias	Meaning
D / B	day / business day
W	week (Sun by default; W-MON etc)
MS / ME	month-start / month-end (pd ≥2.2: "M" deprecated)
QS / QE	quarter
YS / YE	year (was "AS/A")
h / min / s / ms	lower time units (lowercase in pd ≥2.2)

rolling / expanding / ewm

```
df["ma7"] = df["y"].rolling(7, min_periods=1).mean()
df["exp"] = df["y"].expanding().mean()
df["ewm"] = df["y"].ewm(halflife=5).mean()
# BEWARE leakage: rolling uses PAST + CURRENT. For features: .shift(1).rolling(7)
```

shift / diff / pct_change

```
df["y_lag1"] = df["y"].shift(1)
df["dy"] = df["y"].diff()
df["ret"] = df["y"].pct_change()
df["log_ret"] = np.log(df["y"] / df["y"].shift(1))
```

Timezone + slicing

```
df.tz_localize("UTC").tz_convert("America/New_York")
df.loc["2026-01":"2026-06"]      # date-string slicing (inclusive!)
df.loc[df.index.dayofweek < 5]      # weekdays
```

● Missing data (NA / NaN / NaT)

```
df.isna().sum()      # per column
df.dropna(subset=["x"])      # drop rows where x is NA
df.fillna(0); df.fillna(method="ffill")      # NB: method= deprecated → use .ffill() / .bfill()
df.ffill(limit=3)      # forward-fill up to 3 consecutive
df.interpolate(method="time")      # for DatetimeIndex
```

```
# Modern nullable dtypes preserve missingness explicitly:
df["x"] = df["x"].astype("Int64")      # capital I: nullable int
df["s"] = df["s"].astype("string")      # nullable string
# PyArrow-backed for large scale:
df = df.convert_dtypes(dtype_backend="pyarrow")
```

Trap: fillna(method="ffill") deprecated since pd 2.1 → use df.ffill().

● IO — read/write

```
# Parquet — RECOMMENDED (columnar, typed, compressed, fast)
df.to_parquet("data.parquet", engine="pyarrow",
compression="snappy")
pd.read_parquet("data.parquet", columns=["a","b"]) # column
pruning = huge speedup
```

```
# CSV — tolerated but verbose (no dtypes, no schema)
df.to_csv("data.csv", index=False)
pd.read_csv("data.csv", parse_dates=["dt"],
dtype={"id":"int32"})
```

```
# SQL — SQLAlchemy engine
from sqlalchemy import create_engine
e = create_engine("postgres://user:pw@host/db")
pd.read_sql("SELECT * FROM trades WHERE dt > '2026-01-01'", e,
parse_dates=["dt"], chunksize=100_000)
df.to_sql("trades", e, if_exists="append", index=False,
method="multi")
```

```
# Excel — for humans; slow. Multi-sheet:
with pd.ExcelWriter("out.xlsx") as w:
    df1.to_excel(w, sheet_name="A"); df2.to_excel(w,
sheet_name="B")
```

▲ Method chaining (readable pipelines)

```
out = (df
.rename(columns={"amt":"amount"})
.query("region in ['EU','US'] and status == 'settled'")
.assign(month=lambda d: d["dt"].dt.to_period("M"),
gross=lambda d: d["amount"] * d["qty"])
.groupby(["month","region", observed=True)
.agg(total=("gross","sum"), n=("gross","size"))
.reset_index()
.sort_values(["month","total"], ascending=[True, False])
)
```

Use .assign(x=lambda d: ...) so the LAMBDA sees the CURRENT chain state.
Direct expressions read the ORIGINAL df.

Debug chains: .pipe() + .loc[:] pass-through

```
def peek(d, label): print(label, d.shape); return d
out = (df.query("...")
.pipe(peek, "after query")
.groupby("g").sum()
.pipe(peek, "after agg"))
```

◆ Dtypes + memory + Arrow backend

Family	Example	When
numpy int	int32/int64	default; NaN forces float64
nullable int	Int64 (capital I)	needs missingness in ints
float	float32/float64	float32 halves memory
category	"category"	low-cardinality strings; huge speedup
string / string[pyarrow]	"string"	modern; enables .str vec ops
datetime64[ns]	datetime	tz-aware via tz_localize
PyArrow-backed	dtype_backend="pyarrow"	large data · nullable everywhere · fast str

```
df.memory_usage(deep=True).sum() / 1e6 # MB
# Aggressive downcast:
df["x"] = pd.to_numeric(df["x"], downcast="float")
df["region"] = df["region"].astype("category")
```

◆ matplotlib · fig/ax pattern

```
import matplotlib.pyplot as plt

# ALWAYS explicit fig, ax — never rely on plt.state
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(df.index, df["y"], label="observed", lw=1.2)
ax.plot(df.index, df["y_hat"], label="model", ls="--")
ax.set_xlabel("date"); ax.set_ylabel("value")
ax.set_title("Fit vs actual"); ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("fit.png", dpi=150);
plt.close(fig)
```

```
# Grid of subplots
fig, axes = plt.subplots(2, 3, figsize=(14, 6), sharex=True)
for ax, col in zip(axes.flat, df.columns[:6]):
    ax.hist(df[col].dropna(), bins=40); ax.set_title(col)
fig.tight_layout()
```

```
# Twin y-axis (two scales)
fig, ax = plt.subplots()
ax.plot(df.index, df["price"], color="C0");
ax.set_ylabel("price")
ax2 = ax.twinx(); ax2.plot(df.index, df["volume"], color="C1");
ax2.set_ylabel("vol")
```

pandas .plot (matplotlib-backed shortcuts)

```
df["y"].plot(kind="line", ax=ax) # returns the ax
df.plot.scatter(x="x", y="y", c="cat", cmap="tab10")
df.plot.bar(stacked=True); df.hist(bins=30, figsize=(12, 8))
df.boxplot(column="x", by="region") # by= grouping
```

Prefer the fig/ax pattern for anything you'll save or reuse. Use .plot() shortcuts for quick EDA only.

◆ Performance patterns

- **NEVER iterate rows** — vectorised ops are 100-1000x faster
- **Category dtype** — 10x memory reduction + faster groupby on strings
- **Filter early** — df.query() before any groupby
- **Column selection** — read_parquet(columns=[]) for column pruning
- **eval / query** for large frames — numexpr backend, avoids intermediates
- **PyArrow backend** — dtype_backend="pyarrow"; faster strings, native nullable
- **Sort once**; keep sorted (index-based ops assume sorted)
- **Chunked read** — read_csv(chunksize=100_000) or read_sql(chunksize=)

◆ Testing (asserts + fixtures)

```
import pandas as pd
from pandas.testing import assert_frame_equal,
assert_series_equal, assert_index_equal
```

```
assert_frame_equal(a, b, check_dtype=False, check_exact=False,
rtol=1e-5)
assert_series_equal(a["x"], b["x"], check_names=False)
# Test that operations don't mutate input:
before = df.copy()
transform(df); assert_frame_equal(df, before) # ← input
unchanged
```

```
# Fixture pattern (deterministic small df for pytest):
def make_df():
    return pd.DataFrame({"x": [1, 2, 3], "y": [0.1, 0.2, 0.3]},
index=pd.date_range("2026-01-01",
periods=3, freq="D"))
```

▲ Top traps (6-month memory)

- **Chained assignment** df["a"][mask] = v — silently no-ops in pd 3.x (CoW). Use df.loc[mask, "a"] = v.
- **SettingWithCopyWarning** — was pd <2.x; CoW eliminated it. If you see it, you're on legacy pandas.
- **inplace=True** — inconsistent, disables chaining, no perf win. Prefer reassignment.
- **df.append()** — REMOVED in pd 2.x. Use pd.concat([df, row]).
- **pd.get_dummies in prod** — train-vs-serve column drift. Use sklearn OneHotEncoder.
- **Datetime freq "M"** — deprecated in pd ≥2.2. Use "ME" (month-end) or "MS" (month-start).
- **Hour "H"** — deprecated → lowercase "h". Same for "T"→"min", "S"→"s".
- **fillna(method="ffill")** — deprecated. Use df.ffill() / df.bfill().
- **Rolling leakage** — df["y"].rolling(7).mean() includes CURRENT row. For features: .shift(1).rolling(7).mean().
- **Groupby on category without observed=True** — creates rows for unused combinations; huge memory blow-up.
- **Merging on unsorted DatetimeIndex** for merge_asof — silently wrong. Sort both sides first.
- **Reading CSV without dtype** — pandas infers; dates become object, ints become float on any NaN.
- **df.values in new code** — use .to_numpy(); .values is legacy + type-unsafe.
- **Timezone naïveté** — mixing naïve + tz-aware raises. Localize on read.
- **Copy vs view uncertainty** — df.copy() if you need a guaranteed independent frame.

◆ Quick-reference (recall in 5 seconds)

Task	Canonical pattern
Enable CoW	pd.options.mode.copy_on_write = True
Read parquet	pd.read_parquet(path, columns=[...])
Read CSV (typed)	pd.read_csv(p, parse_dates=[...], dtype={...})
Row/col select	df.loc[mask, ["a","b"]]
Filter	df.query("a > 0 and b in ['EU','US'])")
Assign new col	df.assign(x=lambda d: d.a * 2)
Group + agg	df.groupby(k, observed=True).agg(name=("col", "sum"))
In-group feature	df.groupby("g")["x"].transform("mean")
Long → wide	df.pivot_table(index=, columns=, values=, aggfunc=)
Wide → long	df.melt(id_vars=[...], value_vars=[...])
SQL join	pd.merge(a, b, on="k", how="left")
As-of join	pd.merge_asof(a.sort_values("ts"), b.sort_values("ts"), on="ts", by=)
Resample	df.resample("D").agg({...})
Lagged mean	df["y"].shift(1).rolling(7).mean()
Nullable int	df["x"].astype("Int64")
Category	df["c"].astype("category")
Forward fill	df.ffill(limit=3)
To numpy	df["x"].to_numpy() (not .values)
Persist	df.to_parquet(path, compression="snappy")
matplotlib canonical	fig, ax = plt.subplots(); ax.plot(...); fig.savefig(...)